

[Open in app](#)**Medium**

Search



Write



Event-Driven Ansible (EDA) framework for SREs



Gorti Sai Brahmanandam · 5 min read · Jun 9, 2026



1



Of all the declarative tools, my favorite is always Ansible — for the simple reasons, clean and easy to write code with less effort in a reasonable time frame.

I am an automation junkie, and I love writing code, regardless of the language.

That interest has pushed me to learn many languages, starting from shell, Perl, Ruby, and Python.

I did a Golang project, though I hate to put it under scripting languages.

Among all, the scripting language I enjoy writing more and more is Ansible.

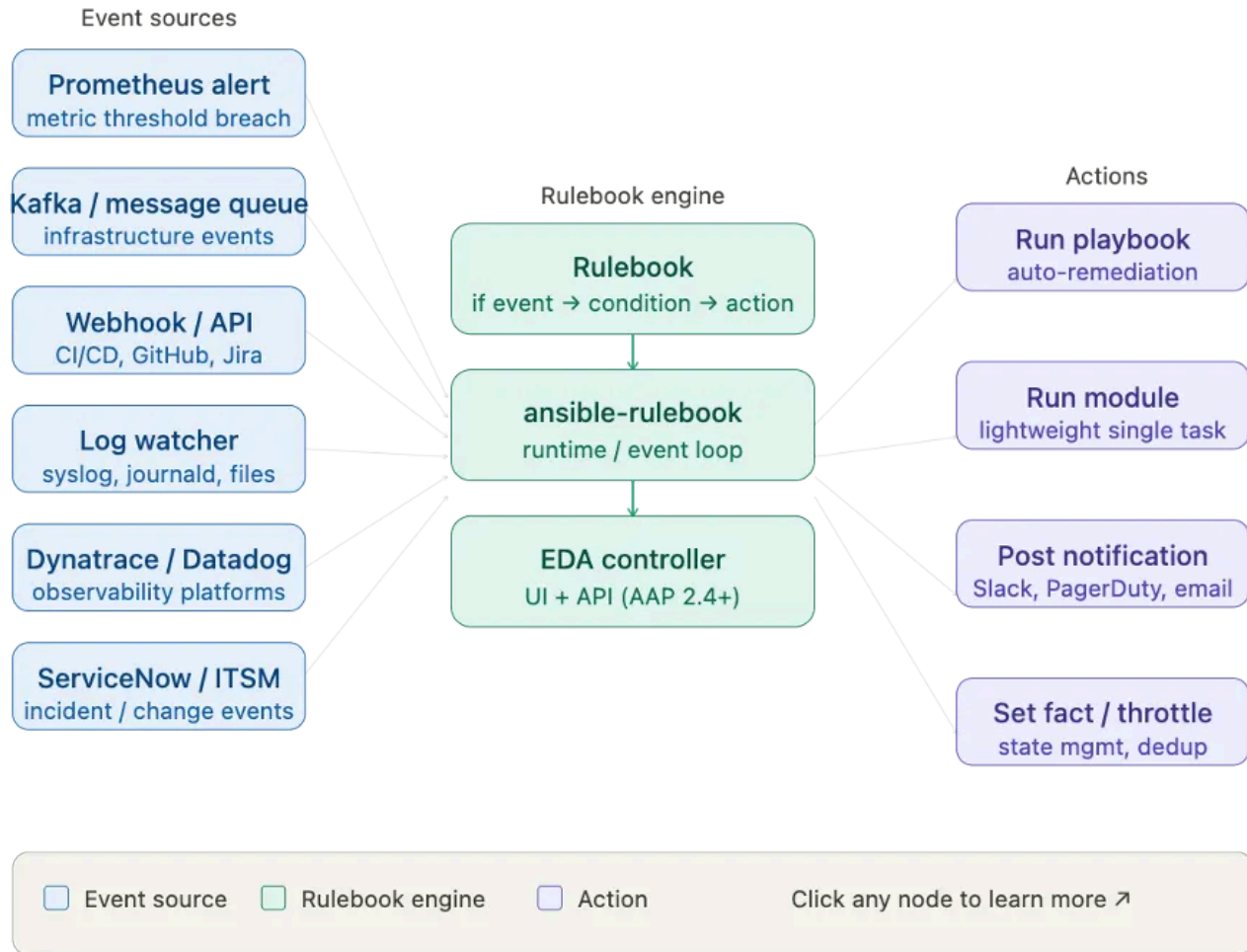
I have had the opportunity to write/ run many Ansible scripts — or playbooks — as part of my DevOps/SRE role. It was simply fun and educational. I loved

the framework that Ansible comes with, and even now, I recollect and cherish the work I did.

In this article, I am not going to emphasize the Ansible scripting or runbooks, but rather introduce something new — Event-Driven Ansible (EDA).

Event-Driven Ansible (EDA) is a framework that lets Ansible react to events automatically — instead of you running a playbook manually, the system listens for something to happen (an alert, a webhook, a log entry), and triggers the right automation response without human intervention.

The core idea is: **if this event → then run that action**



The EDA Controller is the production-grade wrapper that solves all of that. It ships as part of AAP 2.4+ and integrates natively with the Automation Controller (what used to be Ansible Tower).

The six things it adds

Rulebook activations — the core object in EDA Controller. An activation is a running instance of a rulebook, backed by a container. You can start, stop, restart, and monitor them from the UI or API. Each activation runs in its own decision environment (a container image with the rulebook, its Python dependencies, and the `ansible-rulebook` binary baked in).

Web UI + REST API — a browser-based interface to manage everything: create activations, view running events, inspect rule matches, check action

history. The API lets CI/CD pipelines or scripts control EDA programmatically.

Credential management — secrets (Slack webhooks, API tokens, vault passwords) are stored in AAP's credential store and injected into activations at runtime, never hardcoded in rulebooks.

Audit log — every event received, every condition evaluated, every action triggered is recorded with a timestamp. Critical for compliance and postmortems.

Decision environments — analogous to Execution Environments in Automation Controller. You build a container image that contains your rulebook's Python source plugin dependencies, and EDA Controller runs your rulebook inside that image. This makes rulebooks reproducible and portable.

RBAC + teams — inherited from AAP. You control who can create activations, who can view audit logs, which teams own which rulebooks, all through the same role-based access model used for playbooks and job templates.

The three core components:

1. **Event sources** — plugins that listen for things happening. Officially supported ones include *prometheus.eda*, *ansible.eda.kafka*, *ansible.eda.webhook*, *ansible.eda.alertmanager*, and community plugins for *Dynatrace*, *Datadog*, *ServiceNow*, and more.
You can also write your own in Python.

2. **Rulebooks** — YAML files that define the if → then logic. A rulebook has one or more rules, each with a condition (matching on event data) and an

action. Here's a minimal example:

yaml

- **name: Restart service on failure**

hosts: all

sources:

- **ansible.eda.alertmanager:**

host: 0.0.0.0

port: 9000

rules:

- **name: Restart nginx if down**

condition: event.alert.labels.alertname == "NginxDown"

action:

run_playbook:

name: playbooks/restart_nginx.yml

3. Actions

Event-Driven Ansible (EDA) — Actions Reference		
Action	What it does	When to use
<code>run_playbook</code>	Runs a full Ansible playbook	<i>Complex multi-step remediation (e.g. restart service, verify, notify)</i>
<code>run_module</code>	Runs a single Ansible module (lighter weight)	<i>Quick targeted fixes — restart a process, set a file permission</i>
<code>run_job_template</code>	Triggers an AAP job template	<i>Enterprise environments using Automation Controller; auditable job runs</i>
<code>post_event</code>	Sends a new event into the rulebook loop	<i>Chaining rules — one event triggers logic that fires another event</i>
<code>set_fact</code>	Stores state between rule evaluations	<i>Tracking counters, flags, or intermediate values across rules</i>
<code>throttle</code>	Deduplicates noisy repeated events	<i>Preventing alert storms — suppress same alert firing every 30 seconds</i>
<code>shutdown</code>	Gracefully stops the rulebook	<i>Controlled teardown after a task is complete or a terminal error occurs</i>

Where the engines run?

There are two ways to run EDA:

ansible-rulebook is the open-source CLI you install directly — good for testing and self-hosted setups.

The EDA Controller is the enterprise UI/API baked into Ansible Automation Platform (AAP) 2.4+, giving you a **web interface**, **activation management**, **credential handling**, and **audit logs**.

To sum up and recap the key points:

It wraps `ansible-rulebook` in a production-grade control plane with a web UI, REST API, and audit logging

- **Rulebook activations** are the core objects— each runs in its own container (decision environment)
- Credentials are stored securely in AAP's vault, never hardcoded

The critical integration is *run_job_template* — EDA detects the event, Automation Controller runs the playbook

- Adds RBAC and team access control inherited from AAP

Why it matters for SRE work?

The pattern maps directly to the **SRE** toil-reduction principle — instead of an on-call engineer — waking up at 3am to restart a service, scale a deployment, or rotate a certificate — the rulebook handles it automatically the moment the alert fires.

- *Events* become the *trigger*

- *playbooks* become the *response*

- *EDA* is the *wiring* between them

Scenario:

Prometheus detects nginx is down



AlertManager fires webhook



EDA rulebook matches the alert



Ansible playbook auto-restarts nginx

Real Example:

Prometheus (nginx_up == 0 for 1m)



↓ fires alert



AlertManager



↓ POST webhook to EDA port 9000



ansible-rulebook (nginx_remediation.yml)|



↓ matches rule: status==firing AND alertname==NginxDown



restartnginx.yml **playbook** runs automatically



↓ systemd restart → health check → log outcome



notify_slack.yml



↓ SUCCESS or FAILED message to Slack

What each file does?

File	Role in the diagram
<code>nginx_remediation.yml</code>	The rulebook — 4 rules covering firing, warning, resolved, and throttle
<code>restart_nginx.yml</code>	Auto-remediation — logs, restarts, verifies, notifies
<code>notify_slack.yml</code>	Generic Slack notifier called by any rule
<code>inventory.yml</code>	Target hosts EDA hands to the playbooks

With this kind of setup, SREs and other team members get the alert notifications right away. Further, they can extend the logic to suit their corresponding environments.

This is a kind of automation SREs can do easily and extend to other tools as well.

Note:

I am unable provide the source code for the above files. Any SRE can write the code for these files. If anyone has difficulty in writing these files, leave a comment or message me in LinkedIn — I will provide the basic files.



Written by Gorti Sai Brahmanandam

11 followers · 7 following

Edit profile

